

How control logic is generated in IF stage

We begin with *stage_decode()* (inside *pipeline.c*), the control signals are generated by calling *gen_control(instr)* (defined in *stage_helpers.h*), which switches on *instruction.opcode*:

```
idex_reg_t gen_control(Instruction instruction)
```

```
{
    idex_reg_t idex_reg = {0};
    switch(instruction.opcode) {
        case 0x33: // R-type
            idex_reg.reg_write = true;
            idex_reg.alu_src = false;
            break;
        case 0x13: // I-type ALU
            idex_reg.reg_write = true;
            idex_reg.alu_src = true;
            break;
        case 0x03: // Load
            idex_reg.reg_write = true;
            idex_reg.alu_src = true;
            idex_reg.mem_read = true;
            idex_reg.mem_to_reg = true;
            break;
        case 0x23: // Store
            idex_reg.alu_src = true;
            idex_reg.mem_write = true;
            break;
        case 0x63: // Branch
            idex_reg.branch = true;
            break;
        case 0x37: // LUI
            idex_reg.reg_write = true;
            idex_reg.alu_src = true;
            break;
        case 0x6F: // JAL
            idex_reg.reg_write = true;
            idex_reg.jump = true;
            break;
        default: break; // ecall, undefined
    }
}
```

```

return idex_reg;
}

```

Moreover with the implementation we can produce the control signals *reg_write*, *mem_read*, *mem_write*, *mem_to_reg*, *alu_src*, *branch*, and *jump* which are all stored into *idex_reg_t* struct and forwarded through the pipeline procedure. Next *stage_decode()* reads the registers for certain registers *rs1_val/rs2_val* and computes the immediate *imm* via *gen_imm()*, building the full datapath on top of these control bits.

How is the ‘mux’ shown in the IF stage implemented?

Within the *stage_fetch()* function:

```

regfile_p->PC = pwires_p->pcsrc ? pwires_p->pc_src1 : pwires_p->pc_src0;
pwires_p->pcsrc = false;

```

Note that this is a standard C ternary operation in our required context acting as the 2:1 multiplexer that selects our PC (Program Counter): *pcsrc* is the select line in this context, *pc_src0* is the default "PC+4" path (set at the bottom of the previous fetch stage), and *pc_src1* is the redirected target written by decode (for jal instructions) or executes (for branches taken) during the previous cycle. Furthermore, immediately after consuming the instruction, *pcsrc* is then to be cleared such that later stages in the same cycle can start a fresh redirection for the next cycle without collision. Allowing for less hazards and times.

How is the ‘mux’ shown in the EX stage implemented?

Within the *stage_execute()* function:

```

uint32_t alu_inp2 = idex_reg.alu_src ? idex_reg.imm : idex_reg.rs2_val;

```

This is the ALUSrc 2:1 multiplexer with the control input being *alu_src* (set by *gen_control* in ID stage), that choose either the immediate generation (*imm*, for I-type/loads/stores/lui) or the register value *rs2_val* (for R-type/branches).

A table of the ‘alu_op’ values based on the instruction

OPcode	Instruction Type	Additional decoding
0x33	R-type	Funct3 + funct7
0x13	I-type	Funct3 + (imm bit 10 for srai/srli)
0x03	Load	Funct3 (in stage_mem - memory stage)
0x23	Store	Funct3 (again in stage_mem)
0x37	LUI	Forced ADD
default	branch/jal/ecall	Forced ADD (we don't use this result)

Note that the design used doesn't use an independent *alu_op* signal that a standard MIPS datapath would; moreover, by going straight from opcode+ funct3+funct7 to the final *alu_control* code within one function, *gen_alu_control()*. Thus, there isn't an immediate ALUop table to be shown, furthermore, opcode independently acts as that role by using the *switch* structure.

A table of 'alu_control' values based on the instruction

From *gen_alu_control()* in *stage_helpers.h*:

Opcode	funct3	funct7 / imm[10]	Instruction	ALU Operation	alu_control (name / hex)
0x33	0x0	0x00	add	add	ALU_ADD (0x0)
0x33	0x0	0x01	mul	multiply (lower 32b)	ALU_MUL (0x2)
0x33	0x0	0x20	sub	subtract	ALU_SUB (0x1)
0x33	0x1	—	sll	shift left logical	ALU_SLL (0x6)
0x33	0x2	—	slt	set less than (signed)	ALU_SLT (0x9)
0x33	0x3	—	sltu	set less than (unsigned)	ALU_SLTU (0xA)
0x33	0x4	—	xor	XOR	ALU_XOR (0x5)
0x33	0x5	0x00	srl	shift right logical	ALU_SRL (0x7)
0x33	0x5	0x20	sra	shift right arithmetic	ALU_SRA (0x8)
0x33	0x6	—	or	OR	ALU_OR (0x4)
0x33	0x7	—	and	AND	ALU_AND (0x3)
0x13	0x0	—	addi	add	ALU_ADD (0x0)
0x13	0x1	—	slli	shift left	ALU_SLL

				logical	(0x6)
0x13	0x2	—	slti	set less than (signed)	ALU_SLT (0x9)
0x13	0x3	—	sltiu	set less than (unsigned)	ALU_SLTU (0xA)
0x13	0x4	—	xori	XOR	ALU_XOR (0x5)
0x13	0x5	imm[10]=0	srlr	shift right logical	ALU_SRL (0x7)
0x13	0x5	imm[10]=1	srai	shift right arithmetic	ALU_SRA (0x8)
0x13	0x6	—	ori	OR	ALU_OR (0x4)
0x13	0x7	—	andi	AND	ALU_AND (0x3)
0x03	any	—	load (lb/lh/lw)	address calc (add)	ALU_ADD (0x0)
0x23	any	—	store (sb/sh/sw)	address calc (add)	ALU_ADD (0x0)
0x37	—	—	lui	0 + imm (add)	ALU_ADD (0x0)
default	—	—	branch / jal / ecall	unused (add)	ALU_ADD (0x0)

Note that to implement I-type it uses *instr.itype.funct7*. Since I-type has no funct7 field we need to shift it via $(instr.itype.imm \gg 10) \& 0x1$ such that the immediate's top bit doubles as the acting distinguishing bit.

What is the logic behind the implementation of your `gen_branch` function?

Implementation of `gen_branch`

```
bool gen_branch(Instruction instruction, uint32_t rs1_val, uint32_t rs2_val)
{
    bool taken = false;
    switch(instruction.sbtype.funct3)
    {
        case 0x0: taken = (rs1_val == rs2_val); break;           // beq
        case 0x1: taken = (rs1_val != rs2_val); break;         // bne
        case 0x4: taken = ((sWord)rs1_val < (sWord)rs2_val); break; // blt (signed)
        case 0x5: taken = ((sWord)rs1_val >= (sWord)rs2_val); break; // bge (signed)
        case 0x6: taken = (rs1_val < rs2_val); break;           // bltu (unsigned)
        case 0x7: taken = (rs1_val >= rs2_val); break;           // bgeu (unsigned)
        default: taken = false; break;
    }
    return taken;
}
```

In 32-bit riscV we have six distinct branch conditions, therefore, `gen_branch` directly compares itself using `funct3`. This way it selects an individual branch of the six, rather than maneuvering through the ALU's zero flag. To initiate this process we call it from the function `stage_execute`:

```
if (idex_reg.branch)
{
    if (gen_branch(idex_reg.instr, idex_reg.rs1_val, idex_reg.rs2_val))
    {
        pwires_p->pcsrc = true;
        pwires_p->pc_src1 = idex_reg.instr_addr + idex_reg.imm;
        branch_counter++;
    }
}
```

Note the signed comparisons cascade both operands to `sWord (int32_t)` prior comparisons, whilst, `bltu/bgeu` compare the RAW (Read After Write) unsigned `uint32_t` values directly. This way we can correctly distinguish signed vs. unsigned branches at the C level. Allowing the processor to acknowledge the behaviour of Cpp's operands such as `</>=`.